VitaSense AI

PRODUCT REQUIREMENTS DOCUMENT

AI-Driven Integrated Healthcare Monitoring and
Disease Risk Prediction Using Wearable Sensors and Machine Learning

Version 1.0 | Software-only scope | 16 October 2026

Platform-independent specification for AI coding agents and development teams

Contents

1. 1. Document Purpose
2. 2. Product Summary
3. 3. Goals and Success Criteria
4. 4. Target Users and Use Cases
5. 5. Scope
6. 6. Product Principles and Design Direction
7. 7. Information Architecture and Screens
8. 8. Functional Requirements
9. 9. AI/ML and Research Requirements
10. 10. Data Model (Suggested)
11. 11. Suggested Technical Architecture
12. 12. API Outline (Suggested)
13. 13. Security, Privacy, Accessibility, and Reliability
14. 14. Non-Functional Requirements
15. 15. Error, Empty, and Edge States
16. 16. Testing and Acceptance Criteria
17. 17. Deliverables
18. 18. Assumptions, Dependencies, and Open Decisions
19. 19. Implementation Plan
20. 20. Instructions for Any AI Coding Platform
21. 21. Glossary
22. 22. Final Product Statement

1. Document Purpose

This Product Requirements Document defines the software scope, user experience, functional requirements, data model, AI/ML behavior, system architecture, security expectations, validation approach, and acceptance criteria for VitaSense AI.

The project working title is “AI-Driven Integrated Healthcare Monitoring and Disease Risk Prediction Using Wearable Sensors and Machine Learning.” The software product name is VitaSense AI, with the tagline “Your health. Decoded by AI.”

This document is intended to be platform-independent and can be given to AI coding agents, app builders, UI generators, or development teams. It describes expected behavior and outcomes without requiring a particular coding platform.

2. Product Summary

VitaSense AI is a responsive health-monitoring and decision-support web application. It allows users to maintain a profile, enter or import health measurements and symptoms, review personal trends, receive transparent software-generated risk estimates and alerts, and understand data-quality limitations.

The initial deliverable is software only. It must work with clearly labelled sample/demo data and manually entered or imported digital measurements. It must not require the purchase, assembly, or connection of physical sensors to demonstrate the software.

The product is an educational/research prototype and is not a medical device. It does not diagnose disease, replace a clinician, or promise to prevent illness. Predictions and alerts must be presented as informational estimates with limitations and appropriate safety guidance.

3. Goals and Success Criteria

- Provide a polished, responsive, premium health-tech user experience across desktop, tablet, and mobile.
- Support registration, login, logout, profile management, and user-specific saved records.
- Offer a clear dashboard showing recent readings, data quality, alerts, and key trends.
- Allow users to enter symptoms and receive an appropriately cautious, explainable risk estimate based on an explicitly documented model or ruleset.
- Validate supplied data for missing, malformed, stale, duplicated, implausible, or sudden-change readings.
- Show daily, weekly, and monthly trend analysis and let users inspect the underlying measurements.
- Store user history, measurements, symptom assessments, prediction results, and alert events with timestamps.
- Make the system's limitations, data source, model version, and uncertainty understandable.
- Support research evaluation against suitable baselines using reproducible data splits and metrics.

Suggested MVP success measures: all core user journeys complete without blocking errors; all pages usable at common mobile and desktop widths; user records remain isolated between accounts; invalid inputs produce understandable validation messages; charts handle empty and populated states; demo records are visibly labelled; and every risk output includes a disclaimer and explanation.

4. Target Users and Use Cases

- Individual user: creates an account, maintains profile details, enters measurements and symptoms, reviews trends, and manages alerts.
- Project evaluator/researcher: explores demo data, reviews data-quality flags and model explanations, and evaluates model performance on approved datasets.
- Administrator/support role (optional for MVP): reviews aggregate operational health and handles contact submissions without accessing unnecessary personal health information.

Do not build a clinician portal, hospital integration, emergency dispatch system, or insurance workflow unless separately approved as a future scope.

5. Scope

5.1 In scope — software

- Responsive web application / Progressive Web App (PWA)-friendly experience.
- Authentication and account management.
- Profile and health preferences.
- Dashboard with readings, health summaries, data-quality status, trends, and alerts.
- Manual entry and import-ready handling of digital health measurements; sample/demo data for demonstrations.
- Symptom questionnaire and cautious risk-estimation workflow.
- AI/ML explanation panel showing key contributing inputs, data limitations, and confidence/uncertainty where supported.
- Data validation and trust/quality scoring.
- History, filtering, trend charts, and record details.
- Health alerts and user alert preferences.
- Contact Us / feedback form.
- Backend APIs, persistent database, basic logging, testing, documentation, and local deployment instructions.

5.2 Out of scope — initial release

- Physical wearable devices, sensor purchasing, wiring, firmware, embedded programming, and calibration hardware.
- Claims of physical sensor accuracy or clinical validation based only on software checks.
- Automatic real-time streaming from a physical sensor unless a future integration is separately specified.
- Medical diagnosis, treatment recommendations, medication prescriptions, or guaranteed disease prediction.
- Emergency response, ambulance dispatch, clinician decision-making, or replacement of professional care.
- Paid external APIs, paid AI APIs, or mandatory paid hosting.
- Production-grade regulatory certification, hospital EHR integration, or clinical deployment.

Important distinction: “sensor/data validation” in this scope means validating data supplied to the software—such as missing fields, stale timestamps, duplicates, out-of-range values, and suspicious changes. It does not establish that a physical sensor is accurate.

6. Product Principles and Design Direction

- Visual direction: premium, futuristic health-tech; high visual density where useful without sacrificing readability.
- Suggested aesthetic: dark graphite/navy surfaces, electric mint/cyan and restrained violet accents, gradients, translucent/glass-like panels, subtle glow, polished charts, animated-looking signal motifs, and a distinctive AI/health visual.
- Use strong typography, clear hierarchy, purposeful whitespace, accessible contrast, responsive cards, and consistent spacing.
- Animations must be subtle, purposeful, reduced-motion aware, and never obstruct important health information.
- Design must be consistent across the landing page and authenticated application.
- Use loading, empty, success, warning, error, and offline states—not only ideal populated screens.
- Use clear labels and plain language; never imply that a visual badge or score is a medical diagnosis.

The Figma file is the visual source of truth when provided. Implement the approved frames faithfully rather than replacing them with a generic dashboard. Use reusable components and responsive layouts.

7. Information Architecture and Screens

7.1 Public pages

- Landing page: product purpose, key features, safety disclaimer, and clear sign-in/register calls to action.
- Login page: email/username and password, validation, error feedback, and link to registration.
- Registration page: required fields, password validation, consent acknowledgement, and success feedback.
- Contact Us: category, message, optional reply email, validation, submission confirmation, and privacy-conscious handling.

7.2 Authenticated pages

- Dashboard: latest available readings, recent health summary, data quality/trust indicator, recent alerts, trend preview, and quick actions.
- Profile & Preferences: display name, age/date of birth if required, optional sex-related model input only where justified, units, time zone, preferred alert settings, and account controls.
- Measurements / Data Input: enter readings, timestamps, source, units, and optional notes; import-ready workflow; edit/delete permissions.
- Symptom Assessment: guided questionnaire, duration/severity where appropriate, relevant context, review step, and results page.
- AI Risk Analysis: estimate, explanation, inputs used, model/ruleset version, data quality, uncertainty/limitations, and safety note.
- Health Trends: daily/weekly/monthly filters, selectable measures, charts, date range, and measurement-level drill-down.
- History: searchable/filterable timeline of measurements, symptom assessments, predictions, and alerts.
- Alerts: active/recent alert list, severity, reason, timestamp, acknowledgement/dismissal, and alert preferences.
- Data Quality / Validation details: missing, stale, duplicated, implausible, or sudden-change flags with understandable explanations.
- Settings: units, privacy/data controls, notification preferences, password/account controls where supported.

Navigation should be predictable. Desktop may use a sidebar plus top bar; mobile should use a compact navigation pattern. Ensure keyboard focus and accessible labels.

8. Functional Requirements

FR-01 — Authentication and sessions

- Users can register, log in, log out, and receive understandable validation errors.
- Passwords must be securely hashed server-side; never store plaintext passwords.
- Protected pages and APIs require a valid session/token.
- Users can access only their own private profile, measurements, assessments, predictions, and alerts.
- Include password reset only if an email/reset mechanism is actually configured; otherwise clearly mark it as not enabled rather than faking it.

FR-02 — Profile and preferences

- Users can view and update supported profile fields and preferred measurement units.
- Collect only fields necessary for the stated features; sensitive or demographic fields must have a documented purpose.
- Changes are validated and persisted, with clear success/error feedback.
- Profile data must be used only in the context described to the user.

FR-03 — Measurement entry and storage

- Support a configurable measurement schema rather than hard-coding unsupported sensor capabilities.

- Each record should include user ID, measurement type, value, unit, timestamp, source (manual/import/demo), and optional notes.
- Support the initial agreed health measures only. Candidate measures may include heart rate, SpO_2 , temperature, activity, and other values when data and units are defined; do not display unavailable values as real.
- Validate numeric types, required fields, units, timestamps, and reasonable configurable ranges.
- Permit authorized users to inspect, correct, or delete their own entries; preserve appropriate audit events if required.

FR-04 — Data quality and trust scoring

- Detect missing required values, malformed values, unsupported units, duplicate records, stale timestamps, implausible ranges, and configurable sudden changes.
- Label each issue and explain why it matters in plain language.
- Produce a transparent data-quality/trust indicator based on documented rules; show the component checks rather than presenting an unexplained score.
- Low-quality data must be excluded from model inference or explicitly flagged as a limitation.
- Never interpret a software validation pass as proof of physical sensor accuracy.

FR-05 — Dashboard

- Display the latest available data with timestamp and source.
- Show a concise summary of recent trends and recent alerts.
- Show empty states when there are no records and provide a clear action to add data or load demo data.
- Provide quick links to symptom assessment, add measurement, history, and trends.
- Demo/simulated values must be visibly marked as demo data.

FR-06 — Symptom-based risk assessment

- Provide a guided, configurable symptom questionnaire with clear question wording and appropriate response types.
- Collect only the inputs required by the selected model or ruleset.
- Before submission, let the user review their answers and correct them.
- Return an informational risk category or score only if a defined and evaluated model/ruleset supports it.
- Show the factors that contributed to the result, relevant missing data, model limitations, and when to seek professional advice.
- Do not state or imply that the user has a disease; use language such as “estimated risk signal” or “screening-style result,” not “diagnosis.”
- If no validated model is configured, provide a clearly labelled demo/rules-based result rather than pretending a trained clinical model exists.

FR-07 — AI/ML explanation

- Explain the result in plain language using only inputs actually used by the model.
- Where technically supported, show feature contributions or global/local explanation outputs and uncertainty/calibration information.
- Identify the model version, timestamp, and whether the result used demo or user-entered data.
- Do not fabricate feature importance, confidence percentages, model accuracy, or evidence.
- If the model cannot explain an individual result reliably, state that limitation.

FR-08 — Trend analysis

- Support daily, weekly, and monthly views and a selectable date range.
- Plot actual recorded data points and label units, timestamps, missing intervals, and data source where useful.
- Summaries may include min/max/average and direction of change only when enough valid data exists.
- Distinguish observed historical trends from future predictions.
- Do not fill missing data in a way that makes it appear measured; any interpolation must be explicitly marked.

FR-09 — Alerts

- Generate software alerts from documented thresholds, validated rules, data-quality issues, or model outputs.
- Each alert includes title, reason, severity, creation time, related record, and recommended safe next step.
- Users can acknowledge or dismiss alerts; acknowledgement must not delete the underlying event.
- Allow configuration of supported alert preferences and avoid repeated duplicate alerts for the same event.
- Use cautious wording and state that alerts are informational, not emergency monitoring.
- Do not imply continuous monitoring when the application only processes data on entry/import or during a demo.

FR-10 — History and search

- Show a chronological record of measurements, assessments, predictions, and alerts.
- Support date/type filters and pagination or progressive loading.
- Selecting an entry opens its details, inputs, data-quality flags, and result explanation where applicable.
- Edits/deletions must be restricted to the authenticated owner.

FR-11 — Contact Us

- Provide a form for subject/category, message, and optional reply contact.
- Validate input length and format, protect against spam/basic injection, and show success/failure feedback.
- Do not include private health details in email or logs unless explicitly necessary and consented to.
- If no message-delivery backend is configured, save the submission securely or state that sending is unavailable; do not display a false success.

FR-12 — Demo and onboarding

- Provide a guided first-run state or short onboarding explaining the app, data sources, and non-diagnostic purpose.
- Offer optional demo data so evaluators can explore the UI without entering personal data.
- Clearly label demo accounts, demo measurements, and simulated model outputs.
- Provide a reset/clear-demo option that does not delete real user data.

FR-13 — Privacy and account controls

- Explain what information is collected and why before collecting it.
- Provide data export/deletion controls if feasible for the selected architecture and document any limitations.
- Keep secrets and credentials out of frontend source code and version control.
- Use secure transport in deployment and avoid sensitive information in application logs.

9. AI/ML and Research Requirements

9.1 Proposed research direction

Candidate research contribution: “Quality-Aware Personalized Health Trend Analysis and Alerting.” The differentiating hypothesis is that data-quality checks and user-specific baselines, combined with explainable and uncertainty-aware outputs, may reduce misleading alerts compared with fixed thresholds alone. This is a hypothesis to test, not a proven novelty claim.

9.2 Proposed pipeline

- Input ingestion: accept timestamped records with declared units and source.
- Preprocessing: validate schema, normalize units where supported, identify missingness/duplicates/staleness, and flag suspicious values.
- Quality assessment: calculate transparent rule-based quality flags and an interpretable trust score.
- Baseline estimation: estimate an individual trend only after enough valid historical data is available; otherwise use a clearly labelled population/demo baseline if justified.

- Prediction: call a documented model or explicit baseline ruleset; do not invent clinical model capabilities.
- Explanation: expose supported contributing features, data limitations, and uncertainty.
- Alerting: combine supported thresholds/model outputs with data quality and duplicate suppression.
- Logging/evaluation: record model version, input references, timestamp, result, and evaluation metadata without unnecessary sensitive data.

9.3 Evaluation plan

- Use an appropriate public dataset with a documented licence, provenance, target label, and limitations.
- Compare against simple baselines such as fixed thresholds and a basic model suitable for the selected dataset.
- Use participant-wise or subject-wise train/validation/test separation when repeated records per person exist to reduce data leakage.
- Report task-appropriate metrics: AUROC and AUPRC for classification when appropriate; precision, recall, F1, sensitivity/specificity; calibration/Brier score where relevant; and false alerts per unit time and detection delay for alerting tasks.
- Report confidence intervals or repeated-split variation where feasible and document class imbalance and missing data handling.
- Do not claim clinical validity from a public dataset alone. Results must match the actual experiment and be reproducible.

9.4 Safety constraints

- No unsupported disease-specific claims or medical recommendations.
- No synthetic accuracy metrics presented as real.
- No model output when required inputs are invalid or missing without an explicit warning.
- Show uncertainty or “unable to estimate” when the model is outside its supported input conditions.
- Make clear that the prototype is for education/research and not a substitute for a qualified health professional.

10. Data Model (Suggested)

The following logical entities are recommended. Exact columns may be adapted to the selected database.

Entity	Core fields / purpose
User	id, email/username, password_hash, created_at, status
Profile	user_id, display_name, permitted demographic/profile fields, units, timezone, preferences
Measurement	id, user_id, type, value, unit, measured_at, source, notes, created_at
DataQualityResult	id, measurement_id, checks, flags, score, explanation, checked_at, rules_version
SymptomAssessment	id, user_id, questionnaire_version, answers, created_at
RiskPrediction	id, user_id, assessment_id, output_category/score, explanation, model_version, data_mode, uncertainty, created_at
Alert	id, user_id, type, severity, reason, related_record_id, created_at, acknowledged_at, status
ContactSubmission	id, optional_user_id, category, message, reply_contact_optional, status, created_at
AuditEvent (optional)	id, user_id, action, entity_type, entity_id, timestamp; avoid storing secrets or unnecessary health details

Use foreign keys and indexes where supported. Enforce user ownership at the API/database access layer. Define retention and deletion behavior before production use.

11. Suggested Technical Architecture

Recommended default implementation stack; an alternative platform may be used if it preserves the requirements.

- Frontend: React + TypeScript where practical, Tailwind CSS, a charting library such as Recharts, and an accessible component system.
- Backend: Python FastAPI for REST APIs and model/data-processing endpoints.
- Data layer: PostgreSQL for a multi-user deployment; SQLite may be used for a local prototype.
- ML/data processing: Python, pandas, NumPy, scikit-learn, and only additional libraries justified by the chosen model.
- Authentication: secure server-side password hashing and session/token management; never place secrets in the client.
- Testing: frontend component/flow tests, backend unit/API tests, data validation tests, authorization tests, and end-to-end tests for core flows.
- Deployment: local development first; keep deployment configuration portable and avoid mandatory paid services.

11.1 Logical flow

Browser/PWA → frontend routes and components → authenticated API → validation and quality checks → database and prediction/rules service → explanation and alert generation → dashboard/history/trend visualizations.

11.2 Engineering expectations

- Use environment variables for secrets and provide a safe .env.example without real credentials.
- Include setup instructions, dependency installation, database setup/migrations, run commands, and troubleshooting notes.
- Separate UI components, API client, schemas/types, business logic, model code, and tests.
- Handle loading, empty, validation, server-error, and unauthorized states.
- Do not hard-code secrets, fabricate successful API responses, or silently swallow errors.

12. API Outline (Suggested)

These are suggested REST endpoints, not a requirement to use these exact paths. Protect private routes with authentication and ownership checks.

Method & route	Purpose
POST /api/auth/register	Create account
POST /api/auth/login	Authenticate user
POST /api/auth/logout	End session
GET /api/profile	Read current user's profile
PATCH /api/profile	Update current user's profile/preferences
GET /api/measurements	List own measurements with filters
POST /api/measurements	Create a measurement and run validation
GET /api/measurements/{id}	Read one owned measurement and quality details
PATCH /api/measurements/{id}	Correct an owned measurement
DELETE /api/measurements/{id}	Delete an owned measurement
POST /api/assessments	Submit symptom assessment
GET /api/predictions/{id}	Read owned risk estimate and explanation
GET /api/trends	Return aggregated time-series data
GET /api/alerts	List own alerts
PATCH /api/alerts/{id}	Acknowledge/dismiss an owned alert
POST /api/contact	Submit a contact/feedback message
GET /api/demo/overview	Optional demo data only; never mix demo and real data silently

Return consistent error objects with an understandable message and stable error code. Validate input on the server even if the frontend also validates.

13. Security, Privacy, Accessibility, and Reliability

- Authentication required for private records; enforce object-level authorization on every read/update/delete.
- Hash passwords with a modern password-hashing method; use secure cookies or carefully configured tokens.
- Use HTTPS in hosted environments; protect against common web vulnerabilities and validate all inputs.
- Use least-privilege database credentials, environment variables, dependency updates, and basic rate limiting for authentication/contact endpoints.
- Collect the minimum necessary data; provide a privacy notice and explicit demo-data labelling.
- Do not put health data, passwords, tokens, or full request bodies into logs.
- Support keyboard navigation, visible focus, semantic labels, readable contrast, and screen-reader-friendly chart summaries.
- Respect reduced-motion preferences and maintain usable performance on common mobile devices.
- Back up production data and document recovery expectations if the product is deployed beyond a local demo.

14. Non-Functional Requirements

Area	Requirement
Usability	Core tasks should be understandable without specialist medical or technical knowledge.
Responsive design	Support mobile, tablet, and desktop without horizontal overflow.
Performance	Common dashboard and history views should load promptly for expected demo-scale data; avoid blocking rendering for model work.
Reliability	Show clear failure states and preserve user-entered data when practical.
Maintainability	Use modular components, typed schemas where possible, readable naming, and documented setup.
Security	Protect accounts and ensure per-user data isolation.
Transparency	Label data source, demo status, timestamps, model/ruleset version, and limitations.
Accessibility	Keyboard operation, focus indicators, labels, contrast, and reduced-motion support.
Portability	Run locally with documented commands and avoid mandatory paid services.
Auditability	Version the validation rules and prediction model/ruleset used for each output.

15. Error, Empty, and Edge States

- No measurements: explain how to add data or view demo data.
- Stale measurement: display the last measured timestamp and flag it as stale according to configured rules.
- Invalid/out-of-range value: reject or quarantine it with a clear correction path; do not silently treat it as valid.
- Insufficient history: explain that personal trends/baselines are unavailable until enough valid records exist.
- Model unavailable or unsupported input: show “unable to estimate” with the reason; do not fabricate a result.
- Network/server failure: show retry guidance and avoid false success messages.
- Expired session: request sign-in without exposing private data.
- Empty chart range: show an empty state instead of misleading flat lines.
- Duplicate alert: suppress or group repeated events according to documented rules.

- Demo mode: visually mark all simulated readings and outputs.

16. Testing and Acceptance Criteria

16.1 Functional acceptance

- A user can register, sign in, sign out, and update their profile.
- An authenticated user can add a measurement with a timestamp, source, and unit; invalid input is rejected or flagged.
- One user's private records cannot be accessed by another user through UI or direct API calls.
- The dashboard displays actual saved records or clearly labelled demo data.
- Trend views support daily, weekly, monthly, and selected date ranges and show correct units/timestamps.
- Symptom assessment validates answers and returns either a documented model/ruleset result or a clear unavailable/demo state.
- Each prediction displays its explanation, data source, model/ruleset version, and non-diagnostic disclaimer.
- Alerts show their reason and timestamp and can be acknowledged without erasing history.
- Contact submissions produce truthful success/failure feedback.
- Mobile layouts have no unintended horizontal overflow and essential interactions remain usable.
- Loading, empty, error, and unauthorized states are implemented.

16.2 Data/ML acceptance

- Tests cover missing values, malformed values, duplicate records, stale timestamps, unsupported units, implausible values, and sudden-change flags.
- Quality scoring logic is documented and deterministic for identical inputs and rules versions.
- Invalid/low-quality data cannot silently produce an unqualified risk result.
- Evaluation metrics are computed from a documented test set with leakage-aware splits where appropriate.
- Any accuracy, performance, or novelty statement in the UI/report is supported by actual evaluation results.

16.3 Visual acceptance

- Implemented screens are compared against approved Figma frames.
- Typography, spacing, layout, component states, colors, chart style, and responsive behavior are reviewed.
- Distinctive visual elements are preserved; no accidental replacement with generic template styling.
- Animation does not block interaction and respects reduced-motion settings.

17. Deliverables

- Complete responsive software application implementing the agreed pages and flows.
- Source code with clear project structure.
- Database schema/migrations and safe environment-variable example.
- API documentation and setup/run instructions.
- Demo data with visible labelling and reset procedure.
- Automated tests for core business rules, data validation, authorization, and primary journeys.
- Technical architecture summary and known-limitations document.
- Research evaluation notebook/report or scripts, if ML evaluation is included in the agreed software deliverable.
- PRD-to-implementation checklist indicating completed, partial, and deferred requirements.

18. Assumptions, Dependencies, and Open Decisions

- The first release is software-only and must not depend on physical sensor hardware.

- Measurements may be manually entered, imported digitally, or simulated for demo purposes; the source must be visible.
- The final set of supported health measurements and their units must be confirmed before implementing validation ranges.
- A specific disease-risk task, target label, dataset, and evaluation protocol must be selected before training or claiming performance for a model.
- Model explanations depend on the model and data; only supported explanations may be displayed.
- Email delivery, password reset, external integrations, and hosted deployment require actual configuration and must not be faked.
- Any future hardware connection is a separate integration phase and must define protocol, units, timestamps, quality checks, and device identity.

19. Implementation Plan

Phase	Work	Exit condition
1. Foundation	Inspect existing repository; confirm stack; establish routing, theme, responsive shell, environment setup.	App runs; navigation and shared design system are in place.
2. Core accounts	Registration/login/session, profile, authorization, database schema.	Account flows work and private records are isolated.
3. Data layer	Measurement entry, demo data, validation flags, quality details, history.	Records persist and validation tests pass.
4. Dashboard & trends	Dashboard cards, charts, filters, empty/loading/error states.	Charts match saved records and work responsively.
5. Risk workflow	Symptom questionnaire, model/ruleset integration, explanation, limitations.	Outputs are traceable and safely labelled.
6. Alerts & contact	Alert rules, acknowledgement, preferences, Contact Us.	Alerts and feedback have honest, tested outcomes.
7. QA & research	Visual comparison to Figma, accessibility, security, ML evaluation, documentation.	Acceptance checklist completed; limitations recorded.

20. Instructions for Any AI Coding Platform

Use this PRD as the source of truth. First inspect the existing project and report its framework, current features, and gaps. Do not overwrite a functioning project or generate a disconnected second application.

- Translate each functional requirement (FR-01 through FR-13) into tasks and implement in phases.
- Preserve the approved Figma design and use it as the visual reference when a link or screenshots are supplied.
- Ask only for decisions that materially block implementation; otherwise use the documented defaults and record assumptions.
- Implement real persistence and validation rather than UI-only mock behavior, except where data is explicitly labelled demo.
- Do not invent medical evidence, model performance, clinical validation, or research novelty.
- After each phase, run relevant tests and report files changed, commands run, results, known issues, and remaining requirements.
- Finish with a traceability checklist mapping each PRD requirement to its implementation and tests.

21. Glossary

- Data quality: whether a supplied record is structurally valid, timely, complete, and plausible under configured software rules.

- Trust score: a transparent summary of defined data-quality checks; not a measure of sensor calibration or medical truth.
- Risk estimate: an informational model/ruleset output, not a diagnosis.
- Personal baseline: a summary derived from an individual's own valid historical data after sufficient data has been collected.
- Demo data: simulated or sample records used to explore the interface; never presented as actual measurements.
- Model version: identifier for the specific trained model or ruleset used to produce an output.

22. Final Product Statement

VitaSense AI is a software-only, research-oriented health monitoring and risk-estimation platform designed to organize supplied health data, surface trends, assess data quality, provide cautious explainable risk signals, and present useful alerts. It must remain transparent about data sources and uncertainty, protect user information, and avoid claiming clinical diagnosis or physical sensor accuracy.

23. Dataset Strategy and Training Data Requirements

This version extends the original software-only PRD with a secure, repeatable dataset acquisition and ML training workflow. The software must not blindly download every search result or combine unrelated datasets. Only datasets explicitly approved in a manifest may be downloaded.

23.1 Separate ML tasks

- Wearable signal/data-quality task: use timestamped physiological signals to explore missingness, stale data, signal artifacts, activity context, and data-quality/anomaly rules. Such datasets may not contain disease outcomes.
- Clinical risk classification task: use a labelled clinical dataset only for the specific outcome it actually records. A clinical heart-disease table is not automatically wearable data and does not validate wearable-based prediction.
- Personal trend/alert task: use repeated timestamped measurements and participant identifiers. A one-row-per-person dataset cannot validate longitudinal trend alerts.
- UI demonstration: use synthetic/sample records labelled DEMO. Never present simulated records as live measurements or clinical predictions.

Do not merge datasets just to increase row count. Feature definitions, units, population, collection methods, labels, and outcome definitions must be compatible; otherwise train separate models or use each dataset only for its intended analysis.

23.2 Candidate datasets and sources

Dataset/source	Potential use	Limitations and required checks	Official reference
UCI Heart Disease (ID 45)	Small tabular baseline for heart-disease classification and pipeline testing.	303 instances; clinical variables, not wearable time series. Review missing values and cite the dataset. Too small alone to establish broad clinical validity.	https://archive.ics.uci.edu/dataset/45/heart+disease
PhysioNet Wearable Device Dataset: induced stress/exercise	Wearable physiological signal exploration under structured stress and exercise conditions.	Not automatically a disease-diagnosis dataset. Confirm current access policy, licence, variables, labels, and task fit.	https://physionet.org/content/wearable-device-dataset/1.0.1/
Kaggle datasets (curated allowlist only)	Additional labelled data after checking task, target, source, and licence.	Kaggle hosting does not guarantee clinical quality. Verify exact owner/slug, version, licence, schema, and target before enabling.	https://www.kaggle.com/datasets
PhysioNet catalogue	Search for other physiological datasets if required signals are missing.	Some resources require credentialing, training, data-use agreements, or approval. Never bypass access controls.	https://physionet.org/data/

These are candidates, not an instruction to use every dataset or proof that the resulting model predicts disease. Dataset descriptions and access terms can change.

23.3 Dataset approval checklist

- Record canonical source URL, owner, exact dataset ID/slug, version/date retrieved, and retrieval method.
- Review the dataset card, target definition, population, collection method, units, missing-data notes, and limitations.
- Record licence/terms and confirm research, modification, redistribution, and publication permissions.
- Check for personal or sensitive data. Use de-identified data and comply with access restrictions.
- Check archive size and disk space. Preserve original files as read-only; keep metadata and checksums where practical.

- Create a dataset card: purpose, features, target, source, licence, preprocessing, exclusions, bias/limitations, intended and forbidden uses.
- Mark each manifest entry proposed, approved, disabled, or rejected. Only approved entries may download.

24. Kaggle API Credentials and Secure Setup

Kaggle credentials are secrets. Never paste them into an AI chat, commit them to Git, put them in frontend code, or include them in screenshots or the project deliverable.

24.1 Windows setup

23. Sign in to Kaggle and open https://www.kaggle.com/settings/api.
24. Generate an API token using the current Kaggle settings workflow. Depending on the current CLI version, authentication may use a downloaded kaggle.json file or a token/OAuth method; follow the official docs.
25. For the legacy kaggle.json flow, store it at %USERPROFILE%\.kaggle\kaggle.json. Create the .kaggle folder if needed. Do not store it inside the repository or frontend public folder.
26. In the project's Python environment run: python -m pip install --upgrade kaggle
27. Verify with: kaggle --version. Test with a harmless command such as: kaggle datasets list -s "heart disease".
28. If your installed CLI uses OAuth/token authentication, follow the current official documentation instead of forcing the legacy method.

Official docs: https://github.com/Kaggle/kaggle-cli/blob/main/docs/README.md

Dataset download commands: https://github.com/Kaggle/kaggle-cli/blob/main/docs/datasets.md

24.2 Security requirements

- Add kaggle.json, access_token, .env, private downloads, and local credential folders to .gitignore.
- Do not call Kaggle from browser/frontend code. Downloads belong in a local research pipeline or trusted backend.
- Never ask normal VitaSense users to enter Kaggle credentials; these are developer/researcher setup credentials.
- Never log tokens or print credential-file contents. Revoke/regenerate a token if exposed.
- Do not upload credentials to Antigravity or another AI coding platform. Provide setup instructions, not the secret itself.
- Use a dedicated Python virtual environment and restrict credential-file access where practical.

25. Automated Dataset Download Pipeline

Create a repeatable command that reads an approved manifest and downloads only listed resources. Do not scrape pages, guess dataset IDs, download every search result, or automatically accept unknown licences.

25.1 Suggested project structure

- data/raw/ — original downloaded files, never manually edited.
- data/interim/ — intermediate normalized data.
- data/processed/ — model-ready data.
- data/metadata/ — dataset cards, source URLs, licence notes, versions, retrieval timestamps, checksums.
- configs/datasets.yaml — allowlist of approved datasets.
- scripts/download_datasets.py — controlled downloader.
- scripts/validate_datasets.py — schema/file/label checks.
- scripts/prepare_datasets.py — preprocessing and train/validation/test splits.
- scripts/train_model.py — training entry point.
- models/ — model artifacts and model metadata.

25.2 Example manifest

Save this as configs/datasets.yaml. The Kaggle slug is intentionally a placeholder; replace it only after verifying a real dataset and its licence.

```yaml
datasets:
  - id: uci_heart_disease
    enabled: true
    source: ucimlrepo
    uci_id: 45
    task: tabular_heart_disease_baseline
    licence_reviewed: true
    destination: data/raw/uci_heart_disease
    citation_url: https://archive.ics.uci.edu/dataset/45/heart+disease

  - id: physionet_wearable_stress_exercise
    enabled: false
    source: physionet
    url: https://physionet.org/content/wearable-device-dataset/1.0.1/
    task: wearable_signal_exploration
    licence_reviewed: false
    destination: data/raw/physionet_wearable

  - id: kaggle_approved_example
    enabled: false
    source: kaggle
    dataset_slug: OWNER/VERIFIED-DATASET-SLUG
    task: REPLACE_WITH_DEFINED_TASK
    licence_reviewed: false
    destination: data/raw/kaggle_approved_example
    citation_url: https://www.kaggle.com/datasets
```

25.3 Starter Kaggle downloader

This script downloads only enabled Kaggle entries whose licence/access review is explicitly marked true. It validates the slug and skips placeholders. It is a starter implementation; add checksum verification, retries, dry-run support, and stronger logging before production use.

```python
# scripts/download_kaggle_datasets.py
from pathlib import Path
import re
import subprocess
import sys
import yaml

ROOT = Path(__file__).resolve().parents[1]
MANIFEST = ROOT / "configs" / "datasets.yaml"

def main():
    if not MANIFEST.exists():
        raise SystemExit(f"Missing manifest: {MANIFEST}")
    config = yaml.safe_load(MANIFEST.read_text(encoding="utf-8")) or {}
    items = [x for x in config.get("datasets", [])
             if x.get("enabled") is True and x.get("source") == "kaggle"]
    if not items:
        print("No enabled Kaggle datasets.")
        return

    subprocess.run([sys.executable, "-m", "kaggle", "--version"], check=True)
    for item in items:
        name = item.get("id", "unnamed")
        slug = item.get("dataset_slug", "")
        dest = ROOT / item.get("destination", "data/raw/unknown")
        if item.get("licence_reviewed") is not True:
            print(f"SKIP {name}: licence/access review not approved.")
            continue
        if not re.fullmatch(r"[A-Za-z0-9_-]+/[A-Za-z0-9_-]+", slug):
            print(f"SKIP {name}: invalid or placeholder slug {slug!r}")
```

```
            continue
        dest.mkdir(parents=True, exist_ok=True)
        cmd = [sys.executable, "-m", "kaggle", "datasets", "download",
               "-d", slug, "-p", str(dest), "--unzip"]
        print(f"Downloading {name} ({slug}) to {dest}")
        try:
            subprocess.run(cmd, check=True)
        except subprocess.CalledProcessError as exc:
            print(f"FAILED {name}: exit code {exc.returncode}")
            print("Check credentials, dataset access, terms, network, and disk space.")
            continue
        print(f"Downloaded {name}; run schema and data-quality validation next.")

if __name__ == "__main__":
    main()
```

25.4 Install and run

29. Install dependencies: python -m pip install pyyaml kaggle ucimlrepo pandas scikit-learn
30. Save the manifest and script in the paths shown above.
31. Replace the placeholder Kaggle slug with a verified owner/slug, review its terms, then set enabled: true and licence_reviewed: true.
32. Run: python scripts/download_kaggle_datasets.py
33. Inspect downloaded files and dataset documentation before preprocessing. A successful download does not prove suitability.
34. For UCI Heart Disease, use the official page or documented ucimlrepo loader. For PhysioNet, follow that dataset's current access/download instructions.

The script may require adaptation for the current Kaggle CLI version or dataset-specific access flow. Never bypass access controls.

26. Dataset Validation, Preprocessing, and Training Workflow

35. Inventory files, sizes, formats, sources, versions, licence status, and checksums where practical.
36. Validate columns, types, units, target labels, row counts, timestamps, subject IDs, and duplicate keys. Stop if required schema changes unexpectedly.
37. Report missingness, duplicates, invalid ranges, class balance, outliers, and unit inconsistencies. Keep raw data unchanged.
38. Document cleaning, unit conversion, encoding, scaling, resampling, windowing, and feature extraction. Fit transformations on training data only.
39. Split before fitting transformations. Use subject-wise/group splits for repeated measurements and chronological splits when appropriate for time-dependent tasks.
40. Compare with a simple threshold/rule baseline and an appropriate statistical or ML baseline.
41. Record model type, parameters, random seeds, package versions, dataset version, feature schema, and artifact path.
42. Report task-appropriate metrics: precision, recall, F1, AUROC/AUPRC, calibration/Brier score, false alerts per time, and detection delay where relevant. Do not rely on accuracy alone for imbalanced classes.
43. Store model version, supported task/population, expected features, limitations, evaluation results, and dataset citations with each artifact.
44. Do not connect a model to the UI until validation, evaluation, explanation behavior, error handling, and safety language are reviewed.

27. Dataset Automation and UI Requirements

- Downloads and training are developer/researcher workflows, not normal health-app user features.

- Provide CLI commands or an admin-only research tool for manifest status, approved downloads, validation, preprocessing, training, and logs.
- Show progress, success/failure, skip reason, destination, and version. Never display secrets.
- Do not trigger large downloads or model training on every website startup or login.
- Make downloads idempotent where practical and avoid repeated downloads unless force-refresh is selected.
- Include a dry-run/list mode before download.
- Handle network, credential, disk-space, rate-limit, access-revocation, and licence errors safely.
- Do not update production models automatically without review, validation, versioning, and rollback.

28. Additional Dataset/ML Acceptance Criteria

- A documented manifest contains only explicitly approved entries.
- No API credential appears in source control, frontend bundles, logs, documentation, or distributed files.
- The downloader skips disabled entries and entries without reviewed licence/access status.
- Dataset inventory and validation reports are produced.
- Preprocessing is reproducible from documented raw data versions and configuration.
- Splits prevent subject leakage where repeated records exist.
- The UI never implies a model is trained if its artifact is absent or training failed.
- Each model artifact includes task definition, feature schema, dataset citations, version, and evaluation results.
- Dataset licence and citation obligations are documented for the research paper and project submission.

29. Dataset and API Reference Links

- Kaggle CLI documentation: https://github.com/Kaggle/kaggle-cli/blob/main/docs/README.md
- Kaggle dataset download reference: https://github.com/Kaggle/kaggle-cli/blob/main/docs/datasets.md
- Kaggle API settings: https://www.kaggle.com/settings/api
- Kaggle dataset catalogue: https://www.kaggle.com/datasets
- UCI Heart Disease: https://archive.ics.uci.edu/dataset/45/heart+disease
- PhysioNet wearable stress/exercise dataset: https://physionet.org/content/wearable-device-dataset/1.0.1/
- PhysioNet catalogue: https://physionet.org/data/

30. Final Instruction for AI Coding Platforms — Dataset Integration

Implement dataset acquisition and ML training as a separate, secure research pipeline. First inspect the repository and identify existing models, scripts, datasets, and dependencies. Do not overwrite existing artifacts or invent dataset identifiers. Create a curated manifest, secure Kaggle authentication instructions, a downloader for enabled/approved entries, dataset inventory and validation reports, reproducible preprocessing and subject-aware splits, training/evaluation scripts, and model metadata. Use UCI/PhysioNet/Kaggle sources only after task fit, current access terms, licence, and citations have been reviewed. Never put Kaggle credentials in the website or send them to an AI platform. If a dataset slug, access approval, target label, or licence decision is unknown, leave that entry disabled and request review rather than guessing.